```
!pip -q install rdkit
!pip -q install torch-geometric
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.7/36.7 MB 36.2 MB/s eta 0:00:00
```

```
import os, copy, random, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

from rdkit import Chem
from rdkit.Chem import rdmolops

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

from torch_geometric.data import Data
from torch_geometric.loader import DataLoader
from torch_geometric.nn import GINConv, global_mean_pool

print("torch:", torch.__version__)
try:
    import torch_geometric
    print("torch_geometric:", torch_geometric.__version__)
except Exception as e:
    print("torch_geometric import issue:", e)
```

```
torch: 2.10.0+cpu
torch_geometric: 2.7.0
```

```
# =========================
# Configuration
# =========================
SEED = 42
BATCH_SIZE = 32
LR = 5e-4
WEIGHT_DECAY = 1e-4
NUM_EPOCHS = 80
PATIENCE = 10
N_SPLITS = 5
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

set_seed(SEED)

# =========================
# Data
# =========================
def mol_to_graph(smiles, label):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None or mol.GetNumAtoms() == 0:
        return None

    adj = rdmolops.GetAdjacencyMatrix(mol)
    features = [[atom.GetAtomicNum()] for atom in mol.GetAtoms()]
    x = torch.tensor(features, dtype=torch.float)

    edge_index = torch.tensor(np.array(np.nonzero(adj)), dtype=torch.long)
    y = torch.tensor([float(label)], dtype=torch.float)

    return Data(x=x, edge_index=edge_index, y=y)

print("Loading dataset...")
df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
df = df[["smiles", "HIV_active"]].dropna().reset_index(drop=True)

graphs = []
labels = []
for _, row in df.iterrows():
    g = mol_to_graph(row["smiles"], row["HIV_active"])
    if g is not None:
        graphs.append(g)
        labels.append(int(row["HIV_active"]))
```

```
            labels.append(int(row['hiv_active']))

labels = np.array(labels, dtype=int)
print("Valid graphs:", len(graphs))

# ========================
# Model
# ========================
class MPNN(nn.Module):
    def __init__(self):
        super().__init__()
        nn1 = nn.Sequential(nn.Linear(1, 64), nn.ReLU(), nn.Linear(64, 64))
        self.conv1 = GINConv(nn1)
        self.bn1   = nn.BatchNorm1d(64)

        nn2 = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 64))
        self.conv2 = GINConv(nn2)
        self.bn2   = nn.BatchNorm1d(64)

        self.lin1 = nn.Linear(64, 32)
        self.lin2 = nn.Linear(32, 1)
        self.dropout = nn.Dropout(0.3)

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = F.relu(self.bn1(self.conv1(x, edge_index)))
        x = F.relu(self.bn2(self.conv2(x, edge_index)))
        x = global_mean_pool(x, batch)
        x = self.dropout(x)
        x = F.relu(self.lin1(x))
        return torch.sigmoid(self.lin2(x)).view(-1)

def evaluate(loader, model, loss_fn, threshold=0.5):
    model.eval()
    total_loss = 0.0
    probs, true = [], []

    with torch.no_grad():
        for data in loader:
            data = data.to(DEVICE)
            out = model(data)
            y = data.y.view(-1).float()
            loss = loss_fn(out, y)

            total_loss += loss.item() * data.num_graphs
            probs.extend(out.detach().cpu().numpy().tolist())
            true.extend(y.detach().cpu().numpy().tolist())

    probs = np.array(probs)
    true = np.array(true).astype(int)
    pred = (probs >= threshold).astype(int)

    result = {
        "loss": total_loss / len(loader.dataset),
        "accuracy": accuracy_score(true, pred),
        "precision": precision_score(true, pred, zero_division=0),
        "recall": recall_score(true, pred, zero_division=0),
        "f1": f1_score(true, pred, zero_division=0),
        "roc_auc": roc_auc_score(true, probs) if len(np.unique(true)) > 1 else np.nan,
    }
    return result, true, probs

def train_one_fold(train_graphs, val_graphs, test_graphs, fold_id):
    set_seed(SEED + fold_id)

    train_loader = DataLoader(train_graphs, batch_size=BATCH_SIZE, shuffle=True)
    val_loader   = DataLoader(val_graphs, batch_size=BATCH_SIZE, shuffle=False)
    test_loader  = DataLoader(test_graphs, batch_size=BATCH_SIZE, shuffle=False)

    model = MPNN().to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
    loss_fn = nn.BCELoss()

    best_state = None
    best_val_loss = float("inf")
    wait = 0

    history = {
        "loss": [], "val_loss": [],
        "accuracy": [], "val_accuracy": [],
        "auc": [], "val_auc": []
    }

    for epoch in range(1, NUM_EPOCHS + 1):
        model.train()
        total_loss = 0.0
        tr_probs, tr_true = [], []

        for data in train_loader:
            data = data.to(DEVICE)
            optimizer.zero_grad()

            out = model(data)
```

```
            y = data.y.view(-1).float()
            loss = loss_fn(out, y)

            loss.backward()
            optimizer.step()

            total_loss += loss.item() * data.num_graphs
            tr_probs.extend(out.detach().cpu().numpy().tolist())
            tr_true.extend(y.detach().cpu().numpy().tolist())

        tr_probs = np.array(tr_probs)
        tr_true = np.array(tr_true).astype(int)
        tr_pred = (tr_probs >= 0.5).astype(int)

        train_loss = total_loss / len(train_loader.dataset)
        train_acc = accuracy_score(tr_true, tr_pred)
        train_auc = roc_auc_score(tr_true, tr_probs) if len(np.unique(tr_true)) > 1 else np.nan

        val_metrics, _, _ = evaluate(val_loader, model, loss_fn, threshold=0.5)

        history["loss"].append(train_loss)
        history["val_loss"].append(val_metrics["loss"])
        history["accuracy"].append(train_acc)
        history["val_accuracy"].append(val_metrics["accuracy"])
        history["auc"].append(train_auc)
        history["val_auc"].append(val_metrics["roc_auc"])

        if val_metrics["loss"] < best_val_loss - 1e-4:
            best_val_loss = val_metrics["loss"]
            best_state = copy.deepcopy(model.state_dict())
            wait = 0
        else:
            wait += 1
            if wait >= PATIENCE:
                break

    if best_state is not None:
        model.load_state_dict(best_state)

    test_metrics, _, _ = evaluate(test_loader, model, loss_fn, threshold=0.5)

    fold_result = {
        "fold": fold_id,
        "test_accuracy": test_metrics["accuracy"],
        "test_precision": test_metrics["precision"],
        "test_recall": test_metrics["recall"],
        "test_f1": test_metrics["f1"],
        "test_roc_auc": test_metrics["roc_auc"],
        "epochs_ran": len(history["loss"])
    }
    return fold_result, history

# ==========================
# 5-fold CV
# ==========================
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
indices = np.arange(len(graphs))

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(indices, labels), start=1):
    train_graphs = [graphs[i] for i in train_idx]
    train_labels = labels[train_idx]

    temp_indices = np.array(temp_idx)
    temp_labels = labels[temp_idx]

    val_subidx, test_subidx = train_test_split(
        np.arange(len(temp_indices)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_labels
    )

    val_indices = temp_indices[val_subidx]
    test_indices = temp_indices[test_subidx]

    val_graphs = [graphs[i] for i in val_indices]
    test_graphs = [graphs[i] for i in test_indices]

    print(f"\n===== Fold {fold_id} =====")
    print("Train:", len(train_graphs), "Val:", len(val_graphs), "Test:", len(test_graphs))

    fold_result, history = train_one_fold(train_graphs, val_graphs, test_graphs, fold_id)
    print(fold_result)

    all_fold_results.append(fold_result)
    all_histories.append(history)

results_df = pd.DataFrame(all_fold_results)
display(results_df)
results_df.to_csv("MPNN_fold_results.csv", index=False)
```

```
results_df.to_csv("MPNN_fold_results.csv", index=False)

# ========================
# Summary tables
# ========================
metrics_map = {
    "ACCURACY": "test_accuracy",
    "PRECISION": "test_precision",
    "RECALL": "test_recall",
    "F1": "test_f1",
    "ROC_AUC": "test_roc_auc",
}

summary_rows = []
for metric_name, col in metrics_map.items():
    mean = results_df[col].mean()
    std = results_df[col].std(ddof=1)
    var = results_df[col].var(ddof=1)
    summary_rows.append({
        "Metric": metric_name,
        "Mean": mean,
        "Std": std,
        "Variance": var,
        "Formatted": f"{mean:.3f} ± {std:.3f}"
    })

summary_df = pd.DataFrame(summary_rows)
display(summary_df)
summary_df.to_csv("MPNN_summary_results.csv", index=False)

final_table = pd.DataFrame([{
    "Model": "MPNN",
    "Accuracy": summary_df.loc[summary_df["Metric"] == "ACCURACY", "Formatted"].values[0],
    "Precision": summary_df.loc[summary_df["Metric"] == "PRECISION", "Formatted"].values[0],
    "Recall": summary_df.loc[summary_df["Metric"] == "RECALL", "Formatted"].values[0],
    "F1": summary_df.loc[summary_df["Metric"] == "F1", "Formatted"].values[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"] == "ROC_AUC", "Formatted"].values[0],
}])

display(final_table)
final_table.to_csv("MPNN_final_table.csv", index=False)

# ========================
# Average curves
# ========================

def smooth_curve(values, window=5):
    smoothed = []
    for i in range(len(values)):
        start = max(0, i - window + 1)
        smoothed.append(np.mean(values[start:i+1]))
    return smoothed

def pad_and_average(histories, key):
    max_len = max(len(h[key]) for h in histories)
    arr = []
    for h in histories:
        vals = h[key]
        if len(vals) < max_len:
            vals = vals + [vals[-1]] * (max_len - len(vals))
        arr.append(vals)
    return np.mean(np.array(arr), axis=0)

avg_loss = pad_and_average(all_histories, "loss")
avg_val_loss = pad_and_average(all_histories, "val_loss")
avg_acc = pad_and_average(all_histories, "accuracy")
avg_val_acc = pad_and_average(all_histories, "val_accuracy")

plt.figure(figsize=(8,5))
plt.plot(avg_loss, label="Train Loss")
plt.plot(avg_val_loss, label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MPNN Average Loss Curves Across 5 Folds")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(avg_acc, label="Train Accuracy")
plt.plot(avg_val_acc, label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("MPNN Average Accuracy Curves Across 5 Folds")
plt.legend()
plt.show()
```

```
Loading dataset...
Valid graphs: 2885

===== Fold 1 =====
Train: 2308 Val: 288 Test: 289
{'fold': 1, 'test_accuracy': 0.671280276816609, 'test_precision': 0.6805555555555556, 'test_recall': 0.6666666666
```

```
===== Fold 2 =====
```

```
def smooth_curve(values, window=5):
    smoothed = []
    for i in range(len(values)):
        start = max(0, i - window + 1)
        smoothed.append(np.mean(values[start:i+1]))
    return smoothed

avg_loss = pad_and_average(all_histories, "loss")
avg_val_loss = pad_and_average(all_histories, "val_loss")
avg_acc = pad_and_average(all_histories, "accuracy")
avg_val_acc = pad_and_average(all_histories, "val_accuracy")

avg_loss_s = smooth_curve(avg_loss, window=5)
avg_val_loss_s = smooth_curve(avg_val_loss, window=5)
avg_acc_s = smooth_curve(avg_acc, window=5)
avg_val_acc_s = smooth_curve(avg_val_acc, window=5)

plt.figure(figsize=(8,5))
plt.plot(avg_loss_s, label="Train Loss")
plt.plot(avg_val_loss_s, label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MPNN Average Loss Curves Across 5 Folds")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(avg_acc_s, label="Train Accuracy")
plt.plot(avg_val_acc_s, label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("MPNN Average Accuracy Curves Across 5 Folds")
plt.legend()
plt.show()
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	MPNN	0.663 ± 0.032	0.660 ± 0.033	0.706 ± 0.051	0.681 ± 0.032	0.729 ± 0.042



